

National University of Singapore
School of Computing
CS1101S: Programming Methodology
Semester I, 2026/2027

R5
Linked List Processing

Review:

1. A *l*list of a certain data type is `None` or a pair whose head is of that data type and whose tail is a llist of that data type.

```
# Consider a llist of numbers:
my_llist = pair(1, pair(2, pair(3, None)))
# shorter: llist(1, 2, 3)
```

2. Appending two llists

```
def append2(xs, ys):
    return (ys
            if is_null(xs)
            else pair(head(xs), append2(tail(xs), ys)))
```

3. The higher order llist processing trio

```
def map(f, xs):
    return (None
            if is_none(xs)
            else pair(f(head(xs)), map(f, tail(xs))))

def filter(pred, xs):
    return (None
            if is_none(xs)
            else pair(head(xs), filter(pred, tail(xs)))
            if pred(head(xs))
            else filter(pred, tail(xs)))

def reduce(op, initial, xs):
    return (initial
            if is_none(xs)
            else op(head(xs),
                    reduce(op, initial, tail(xs))))
```

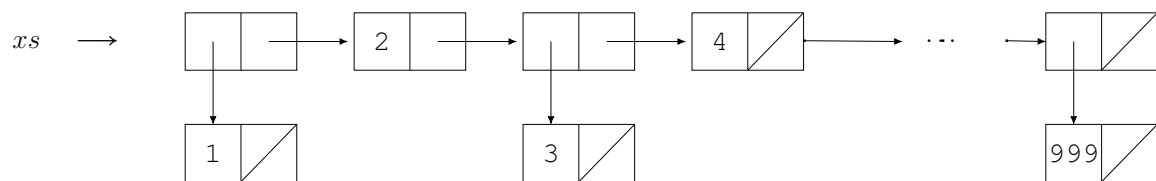
Problems:

1. You are given the `LoN` below, try to produce the following results using only `map/ filter / reduce` function applications.

```
LoN = llist(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

- (a) `llist(2, 4, 6, 8, 10, 12, 14, 16, 18, 20)`
- (b) `llist(1, 1, 3, 3, 5, 5, 7, 7, 9, 9)`
- (c) Sum of even numbers up to and including 10

2. For the following box-and-pointer diagram, write a Source program such that at the end of the evaluation of your program, the name `xs` will have the value as shown in the diagram. You must not use any ellipsis (...) in your program. Hint: You can create a llist of integers from 1 to 999 using `enum_llist(1, 999)`.



3. Consider llists of llists of numbers. Here is an example:

```
my_matrix = llist(llist(1, 2, 3), llist(4, 5, 6), llist(7, 8, 9))
```

Write a function `flatten_llist` **using the function `reduce`**, that “flattens” the given llist of llists of numbers: It returns a llist of numbers that contains the same numbers in the same order as the given llist of llist of numbers. Example:

```
my_llist = flatten_llist(my_matrix)
# returns: llist(1, 2, 3, 4, 5, 6, 7, 8, 9)
```

```
def flatten_llist(lst):
    # your answer here
    ...
```

```
flatten_llist(llist(llist(1, 2, 3), llist(4, 5, 6), llist(7, 8, 9)))
# Returns: llist(1, 2, 3, 4, 5, 6, 7, 8, 9)
```